

Programación Web

Unidad 12: Despliegue y CI/CD

Post-Contenido 1 — Contenedorizar la Aplicación
Spring Boot y Desplegar en Railway

Farid Lobo

Código estudiantil: 1152338

GitHub: github.com/faridl28/lobo-post1-u12

Ingeniería de Sistemas — Universidad Francisco de Paula Santander
2026

Aplicación Desplegada en Railway

<https://lobo-post1-u12-production.up.railway.app>

La aplicación está desplegada públicamente con PostgreSQL conectado. El endpoint **/actuator/health** retorna **{"status":"UP"}** con la base de datos activa.

Endpoints verificados en Railway:

Método	Endpoint	Descripción	Código
GET	/actuator/health	Estado de la aplicación	200
POST	/api/productos	Crear producto	201
GET	/api/productos	Listar productos activos	200
GET	/api/productos/{id}	Buscar por ID	200 / 404
DELETE	/api/productos/{id}	Eliminar producto	204 / 404

Parte 1: Dockerfile Multi-Stage

Se implementó un Dockerfile con estrategia multi-stage para separar el ambiente de compilación (JDK + Maven) del ambiente de ejecución (solo JRE), reduciendo el tamaño de la imagen final. La imagen resultante ocupa **126 MB**, muy por debajo del límite de 300 MB.

Dockerfile

```
# Etapa 1: compilacion con Maven
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
RUN apk add --no-cache maven
COPY pom.xml .
RUN mvn dependency:go-offline -q
COPY src ./src
RUN mvn clean package -DskipTests -q

# Etapa 2: imagen de produccion (solo JRE)
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S spring && adduser -S spring -G spring \
&& mkdir -p /app/logs \
&& chown -R spring:spring /app
USER spring
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

.dockerignore

```
target/
.git/
.idea/
*.iml
.mvn/
*.md
docker-compose.yml
logs/
```

Checkpoint 1

Dockerfile con dos etapas (builder y produccion) creado en la raiz del proyecto. .dockerignore excluye target/ y .git/. Imagen construida con docker build -t catalogo:local . sin errores. Tamano de imagen: 126 MB (inferior a 300 MB).

Parte 2: Perfiles y Docker Compose

Se configuró el perfil de producción **application-prod.properties** que lee las credenciales desde variables de entorno, y el archivo **docker-compose.yml** que orquesta la aplicación junto con PostgreSQL 16.

application-prod.properties

```
spring.datasource.url=${DATABASE_URL}
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASS}
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
logging.level.root=WARN
logging.level.com.empresa.catalogo=INFO
management.endpoints.web.exposure.include=health,info
management.endpoint.health.show-details=always
```

docker-compose.yml

```
version: "3.9"
services:
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      - SPRING_PROFILES_ACTIVE=prod
      - DATABASE_URL=jdbc:postgresql://db:5432/appdb
      - DB_USER=appuser
      - DB_PASS=apppass
    depends_on:
      db:
        condition: service_healthy
    networks:
      - app-net
  db:
    image: postgres:16-alpine
    environment:
      - POSTGRES_DB=appdb
      - POSTGRES_USER=appuser
      - POSTGRES_PASSWORD=apppass
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser -d appdb"]
      interval: 5s
      timeout: 3s
      retries: 5
    networks:
```

```
- app-net
```

```
volumes:
```

```
postgres-data:
```

```
networks:
```

```
app-net:
```

Checkpoint 2

docker compose up -d --build levanta ambos servicios sin errores. docker compose ps muestra app y db en estado Up/healthy. GET <http://localhost:8080/actuador/health> retorna status UP con PostgreSQL conectado. Los tres endpoints REST funcionan correctamente con la base de datos.

Parte 3: Despliegue en Railway

Railway detectó automáticamente el Dockerfile del repositorio y construyó la imagen en la nube. Se aprovisionó una base de datos PostgreSQL y se configuraron las variables de entorno mediante el panel web.

Variables de entorno configuradas en Railway:

Variable	Valor configurado
SPRING_PROFILES_ACTIVE	prod
DATABASE_URL	\${{Postgres.DATABASE_URL}}
DB_USER	\${{Postgres.PGUSER}}
DB_PASS	\${{Postgres.PGPASSWORD}}

Checkpoint 3

Aplicacion desplegada en Railway con URL publica:
<https://lobo-post1-u12-production.up.railway.app>. El endpoint /actuator/health retorna status UP con PostgreSQL conectado. Los tres endpoints REST (GET /api/productos, POST /api/productos, GET /api/productos/1) funcionan correctamente desde la URL de Railway.

Entregables

Elemento	Especificacion
Repositorio GitHub	github.com/faridl28/lobo-post1-u12
URL Railway	lobo-post1-u12-production.up.railway.app
Dockerfile	Multi-stage con JDK builder y JRE produccion, usuario no root
.dockerignore	Excluye target/, .git/, .idea/, logs/
docker-compose.yml	Orquesta app + PostgreSQL 16 con healthcheck
application-prod.properties	Perfil de produccion con variables de entorno
README.md	Instrucciones de ejecucion, variables y URL de Railway
Evidencia 1	docker build exitoso, imagen 126 MB
Evidencia 2	docker compose ps con app y db healthy, actuador/health UP
Evidencia 3	3 endpoints funcionando en URL publica de Railway